

1차시 · 바이브 코딩

바이브 코딩 & Codex 첫걸음

터미널에서 시작하는 AI 페어 프로그래밍.

“마지막으로 코드를 짤 때, AI를 얼마나 썼나요?”

자동완성 → 챗봇에 질문 → 이제는 에이전트에게 위임

SECTION 01

개발의 무게중심이 옮겨졌다

1세대=줄, 2세대=답변, 3세대=작업 AI가 돕는 '단위'가 커졌다.

AI 코딩 도구는 어디까지 왔나



코파일럿식 자동완성(줄/함수) → ChatGPT식 답변(복붙) → 목표를 주면 스스로 일하는 에이전트. 도움의 단위가 줄 → 답변 → 작업으로 커졌다.

같은 'AI 코딩'이지만 결이 다르다

자동완성

- 단위: 줄 / 함수
- 맥락: 현재 파일만
- 실행 권한: 없음
- 보일러플레이트엔 빠름

챗봇 Q&A

- 단위: 답변 한 덩어리
- 맥락: 내가 붙여넣은 것
- 실행 권한: 없음
- 복붙·문맥 전달은 내 몫

에이전트

- 단위: 작업 / 기능
- 맥락: 프로젝트 전체
- 실행: 파일·터미널
- 스스로 모으고 실행·검증

왜 자동완성·챗봇만으론 부족했나

▶ 빠져 있던 것: 스스로 맥락을 모으고 · 실행하고 · 확인하는 주체

단위가 작다

줄·함수는 채워도 '기능 하나'는 통째로 못 만든다

맥락을 모른다

폴더 구조 · 요구사항 · 이전 결정 · 코딩 규칙 바깥

손이 없다

챗봇은 내 파일을 직접 읽지도, 고치지도 못한다

→ 병목은 그대로

설계 · 조율 · 통합 · 복붙은 전부 사람 몫

SECTION 02

에이전트에게 일을 맡긴다

에이전틱 AI = 자율·도구·반복 / 바이브 코딩 = 의도·생성·검증

사람은 What, 에이전트는 How

사람 — What

- 목표를 제시
- 제약·예시를 분명히
- 결과를 검증
- 방향을 수정

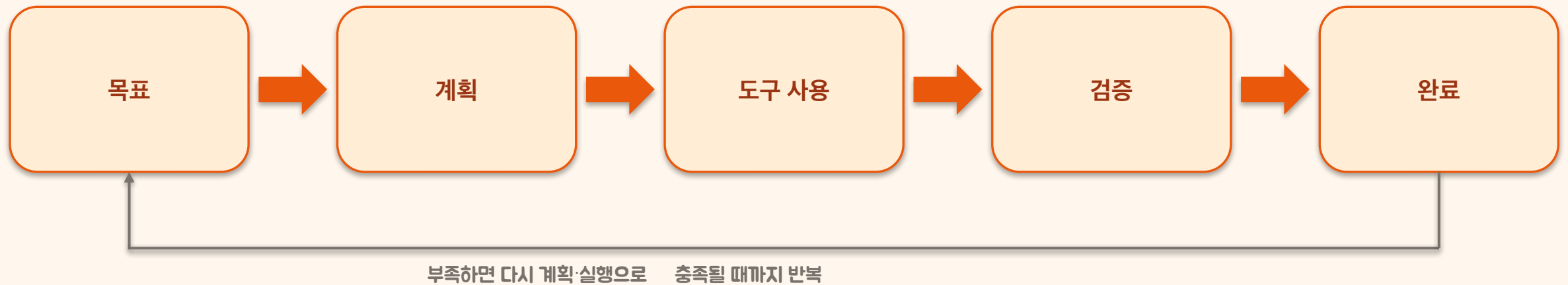
에이전트 — How

- 스스로 계획
- 도구를 사용해 실행
- 코드·파일을 생성
- 목표 달성까지 반복

에이전틱 AI를 이루는 세 가지

- 자율성(Autonomy) — 매 단계 지시 없이 다음 할 일을 스스로 정함
- 도구 사용(Tool use) — 파일 읽기·쓰기 · 명령 실행 · 검색 · (나중에) MCP
- 반복(Iteration) — 한 번에 끝내지 않고 실행→검증→수정을 스스로 돈다
- 이 셋이 합쳐져 '목표만 주면 알아서 일하는' 동작이 가능

목표를 주면 스스로 도는 엔진



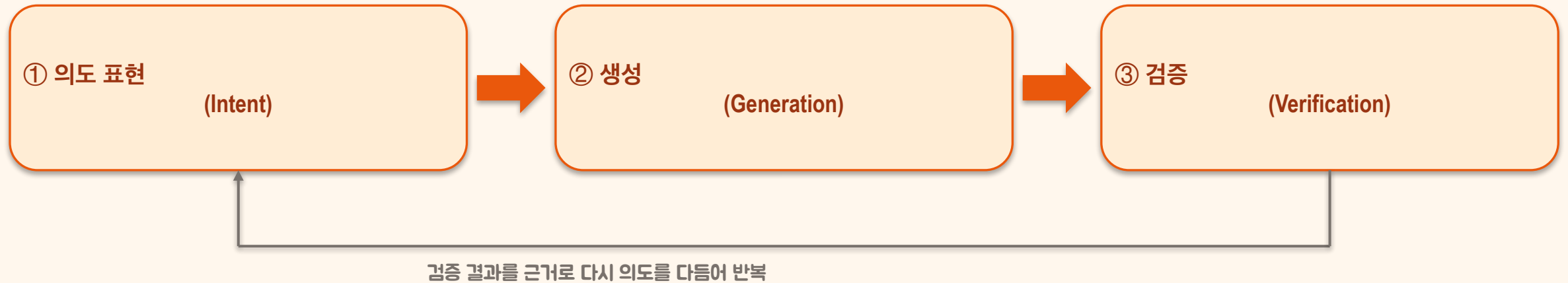
이 루프가 바이브 코딩의 엔진. 사람은 목표 제시와 검증에 개입하고, 계획·실행·재시도는 에이전트가 실행한다.

바이브 코딩이란 무엇인가

개발자와 AI 에이전트의 새로운 소통 방식 자연어로 '무엇을' 전달

- 코드를 직접 치기 → 의도를 명확히 전달하고 결과를 검증하기로 무게중심 이동
- vibe = 의도/분위기 정확한 의도 전달이 곧 실력
- 세 가지 활동의 반복: ① 의도 표현 → ② 생성 → ③ 검증

의도 → 생성 → 검증을 반복한다



의도가 모호하면 결과도 모호하다. 생성은 에이전트 몫, 검증은 사람 몫. 돌려보고 눈으로 확인한다.

이미 바이브 코딩으로 만들어진 것들

15시간

Refetch — HN 대안 뉴스
아이디어 → 완성 "Appwrite"

\$11k/월

Disko — SMS 마케팅
4개월 만의 월매출 "Replit"

17분

Airbnb 클론
"GPT-5 + Lovable"

개인 취미(동화 생성 Storypot, 가족사진 AI영상)부터 수익형 MVP까지. 오늘 만들 점심 추천기도 같은 방식의 작은 출발점.

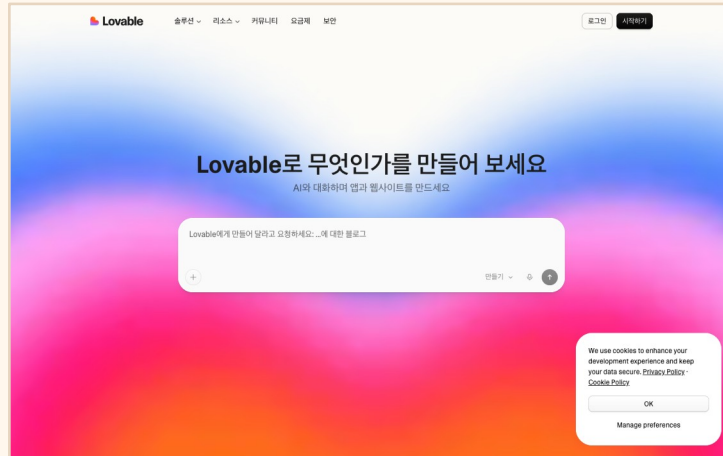
무엇으로, 어떻게 만드나

- 종류: 웹앱 · 게임 · 크롬 확장 · 대시보드 · 자동화 봇 · 3D 사이트 · 리서치 툴
- 대표 빌드 도구: Lovable · Bolt · Replit · v0 · Cursor · Appwrite · Supabase
- 우리 캠프는 이 중 터미널 기반 Codex CLI로 진행 도구는 달라도 원리는 같다
- ⚠ 바이브 코딩은 프로토타입에 특히 강함 → 실서비스 전엔 사람이 코드 검토

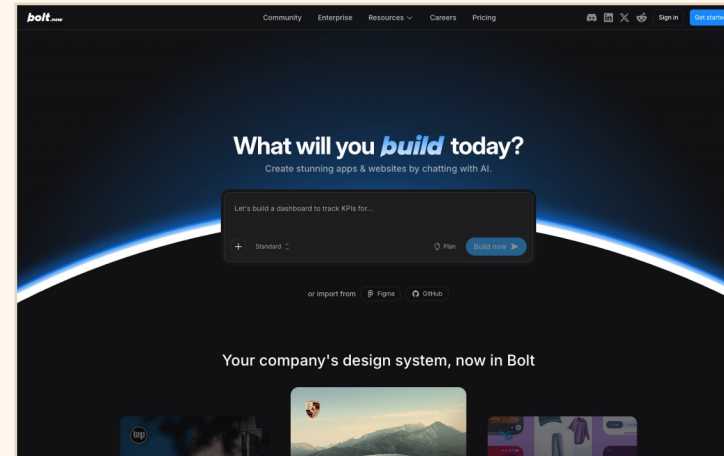
실제 화면

대표 빌드 도구 — 실제 사이트

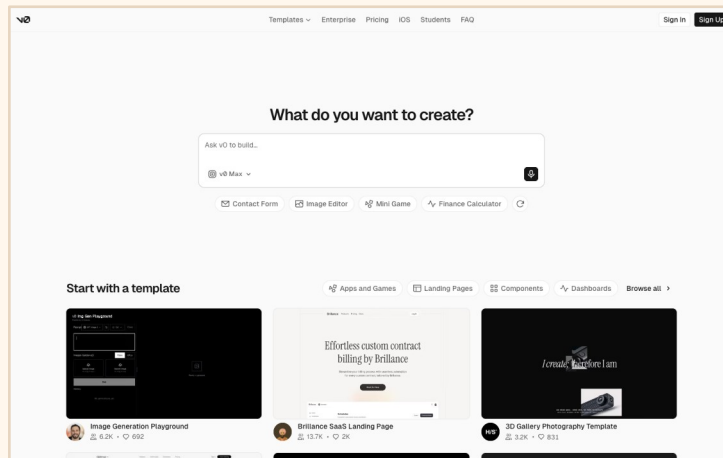
프롬프트 한 줄로 시작하는 빌더들 — 수업 후 직접 들어가 보세요



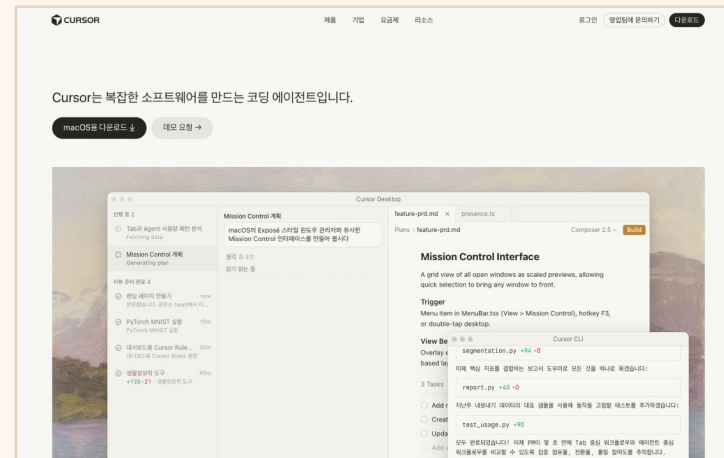
Lovable — lovable.dev



Bolt — bolt.new



v0 — v0.dev



Cursor — cursor.com

SECTION 03

왜 터미널인가

에이전트의 손발이 닿는 작업 공간

에이전트가 일하려면 손이 닿아야 한다

▶ 파일 · 명령 · 도구가 모두 닿는 자리 — 그래서 Codex는 'CLI'



“터미널, 어렵지 않나요?”
→ 명령을 외울 필요 없습니다. 자연어로 말하면 Codex가 명령을
실행합니다.

두려움 제거 오늘 직접 만져봅니다

SECTION 04

Codex CLI와의 첫 인사

Codex CLI란?

- OpenAI의 터미널 기반 코딩 에이전트 선택한 폴더에서 코드를 읽고·고치고·실행
- 설치·계정·키·로그인은 0차시에서 완료
- 빠른 점검: 터미널에서 `codex --version` 이 나오면 OK
- 안 된 분은 0차시 가이드로 먼저 해결

실행부터 검증까지 4단계

1

첫 실행

작업할 폴더로 이동 후 **codex** 실행 → 계정/키로 로그인

2

세션 시작

대화 맥락이 유지되는 단위 이어 요청하면 앞 내용을 기억 (한 세션 = 한 작업 흐름)

3

첫 요청

자연어로 목표 전달 → 에이전트가 파일을 생성·수정 (승인 모드 권장)

4

결과 검증

생성된 파일을 브라우저 에디터로 직접 확인 '생성됐다 ≠ 맞다'

복붙으로 바로 따라하기

터미널에서 폴더로 이동 → codex 실행 → 자연어로 요청

```
cd my-project    # 작업할 폴더  
codex            # 실행 → ChatGPT 계정 또는 API 키로 로그인
```

> 이 폴더에 "Hello"를 보여주는 index.html을 만들어줘.

- 입력 → 파일 생성 → 결과 확인의 한 흐름
- 명령어 암기 불필요 의도를 문장으로 전달하면 된다

막혔을 때

AI도 틀린다 — 중요한 건 고치는 법

▶ 에이전트는 자주 틀린다 — 그게 정상

① 에러를 그대로
붙여넣기



② 한 단계만
다시 요청



③ 작게 쪼개기 ·
문제 지점 지목

🔗 교체 말고 수정 — '막연히 더 좋게'(X) → '이 버튼을 파랗게'(O)

SECTION 05

반복의 기술 — Iteration Loop

작게, 자주, 반복 + 매 단계 검증 = 품질

전통 코딩 vs 바이트 코딩

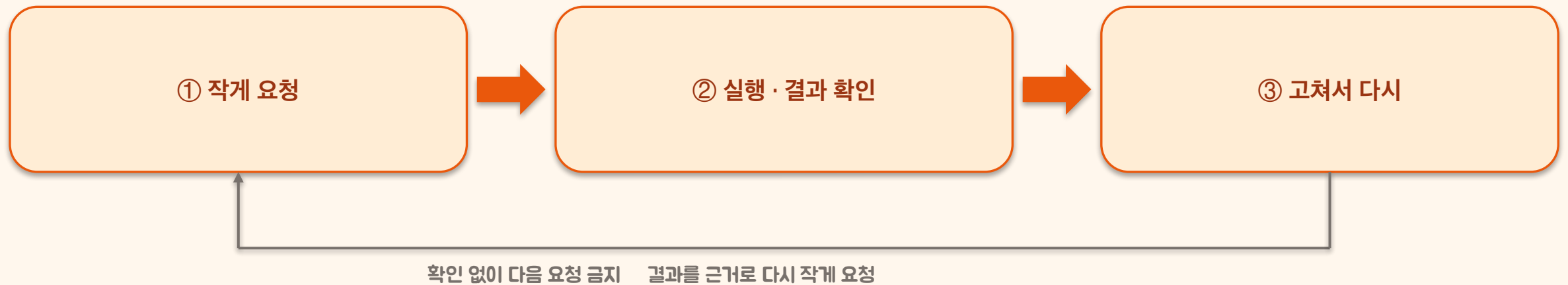
전통 코딩

- 크게 설계 → 구현 → 테스트
- 한 덩어리를 길게 진행
- 문제는 한참 뒤에 발견
- 어디서 깨졌는지 추적이 지옥

바이트 코딩

- 작게 · 자주 · 반복
- 짧은 사이클을 여러 번
- 매 단계 즉시 확인
- 틀린 지점을 금방 안다

한 번에 완성하지 말 것



'전부 만들어줘'(X) → '버튼 하나부터'(O). ②는 사람의 검증 위임해도 책임은 내 것.

작게, 자주 — 그리고 매번 검증

▶ 짧은 루프 = 빠른 피드백 = 적은 디버깅

잘못된 워크플로

- 한 번에 크게 요청
- 한참 뒤에야 확인
- '막연히 더 좋게' 식 수정
- 어디서 깨졌는지 추적 지옥

VS

좋은 워크플로

- 작게 요청 → 즉시 확인 → 반복
- 결과를 근거로 구체적으로 수정
- 틀린 지점을 금방 안다 · 되돌리기 쉬움
- 검증은 언제나 사람의 몫

SECTION 06

실습: 랜덤 점심 추천기

랜덤 점심 추천기 — 스펙

- 버튼을 누르면 메뉴를 랜덤 추천하는 단일 HTML 페이지
- 스펙: 입력 없음 / 버튼 1개 / 클릭 시 메뉴 1개 표시 / 단일 HTML 파일
- UI도 로직도 Codex CLI 하나로 (v0·스티치 등 없음)
- 산출물: 동작하는 `app.html` — '프롬프트만으로 화면이 나온다'를 체감

작게 요청 → 확인 → 고쳐 다시

1

한 프롬프트로 생성

"버튼을 누르면 랜덤으로 점심 메뉴를 추천하는 단일 HTML 페이지를 만들어줘." → 브라우저로 열어 확인

2

iterate: 필터

"한식/양식/중식 필터 버튼을 추가해줘." → 변화 확인

3

iterate: 기록

"최근 추천 5개를 아래에 기록해줘."

4

막히면

에러 그대로 붙여넣기 → 한 단계만 다시 → 작게 쪼개기

SECTION 07

정리 & 다음 차시

오늘부터 코드를 '치는' 사람이 아니라 '지휘하는' 사람

체크포인트 — 다 됐나요?

- 자동완성 → 에이전틱 전환을 설명할 수 있다
- 에이전틱 루프(목표·계획·도구·검증·반복)를 이해했다
- Codex 세션을 실행하고 첫 요청을 해봤다
- 점심 추천기 1단계를 완성했다
- 다음 차시(2차시): 컨텍스트 엔지니어링 AGENTS.md·프롬프트·MCP·스킬로 품질 끌어올리기

목표를 주고, 작게 반복하고, 직접 검증한다.

오늘부터 당신은 코드를 '치는' 사람이 아니라 '지휘하는' 사람